

# Defense notes — Imagining the Target

Asma Hoseinpour Siouki · MSc Data Science · University of Padova · July 2026

Target: ~20

## 1. Title

0:30 · cum 0:30

Good morning everyone, and thank you for being here. My thesis is about a search problem: finding one specific object inside a very large photo collection, when the query is not just an image and not just text — but an image *plus* a text that modifies it. And the constraint that shapes everything: we do it without training any model at all.

## 2. What is Composed Image Retrieval?

1:00 · cum 1:30

Let me start with the task itself. In Composed Image Retrieval — CIR for short — the query has two parts. Here is an example I will keep coming back to during the whole talk: a photo of the Temple of Poseidon, plus the sentence "in an old archival photo". What I want back is *this same temple*, but as it appears in an old archival photograph.

Why is this useful? Think of online shopping — "this bag, but in leather"; or a photo archive — "this building, at night". Neither words alone nor an image alone can express these queries: the image carries *what* you mean, the text carries *how* you want it changed.

## 3. Instance-level retrieval: this exact object

1:00 · cum 2:30

Most existing work on this task is *category-level*: if I ask for "a red dress like this", any red dress is a correct answer. My thesis works on the harder, *instance-level* version: "THE Temple of Poseidon", "THE mask of Agamemnon" — only the very same object counts as correct.

On the right you see the kinds of objects involved — landmarks, products, cartoon characters, artworks. The difficulty is that the same object must be recognized under drastic appearance changes: at night, as a sketch, as a scale model, as a painting. A system that only understands categories has nothing to hold on to here.

## 4. The i-CIR benchmark

1:30 · cum 4:00

All experiments run on i-CIR, a recent benchmark built exactly for this task. Let me walk through one query so the format is clear. The query is the temple image plus a modification text. The green row shows the *positives*: the same temple, under the modified condition. The other rows are what makes the benchmark hard — *designed negatives*: a visually similar temple — right look, wrong identity; the right condition on the wrong object; and combined negatives that get both almost right.

*If they linger: the negatives are curated per query, so random guessing or shortcut features score very poorly.*

The scale: one thousand eight hundred eighty-three queries over two hundred and two object instances, and the search happens in a database of about seven hundred fifty-two thousand images. So this is needle-in-a-haystack retrieval.

## 5. Constraint: no training — zero-shot

1:00 · cum 5:00

One more design decision before the method. Most CIR systems in the literature *\*train\** a fusion network on labelled triplets — image, text, and the correct target. At instance level that is not really an option: there is no training set for these two hundred and two objects, and every new object you would ever want to search for would need new labelled data.

So the whole thesis is *\*zero-shot\**: I assemble large pre-trained models, off the shelf, and add no learned parameters whatsoever. The advantage is immediate generalization — a new instance needs nothing re-trained, ever.

## 6. How we measure success

1:00 · cum 6:00

How do we score such a system? The system returns a ranked list of the whole database. Intuitively: did the true targets come out on top? Average Precision captures exactly that — it is high when the true images sit at the very top of the list. We average it over all queries to get mean Average Precision, mAP.

The main metric is *\*macro\**-mAP: queries are first averaged per object, and then across the two hundred and two objects — so every object counts equally, regardless of how many queries it has.

| *Statistics question likely here → Backup slide 19 has the formulas.*

## 7. The tool: images and text in one space

1:30 · cum 7:30

Now the key tool, and I want to spend a moment here because everything that follows relies on it. CLIP is a pair of neural networks — one reads images, one reads text — trained on four hundred million image-caption pairs from the web, with one objective: an image and a text that *\*mean the same thing\** should land on *\*nearby points\** in a shared space of vectors.

So after CLIP, everything — any photo, any sentence — becomes a point in the same space, and "how similar is this image to this text" becomes a simple geometric question: how close are the two points. That single number is the building block of the whole pipeline. And importantly: CLIP comes pre-trained; I use it as-is.

## 8. The starting point: BASIC

1:30 · cum 9:00

The i-CIR paper proposes a zero-shot method called BASIC, which is my starting point and baseline. The idea in one sentence: score every database image *\*twice\** — does it *\*look like\** the query image, and does it *\*match\** the modification text — and multiply the two similarities.

The multiplication acts like a soft logical AND: an image only scores high if it satisfies *\*both\** conditions. A photo of the right temple in the wrong condition fails the text test; an archival photo of the wrong building fails the image test. There is some careful score normalization inside — I'll spare you the details, they are in the backup — but conceptually it is: image similarity times text similarity.

| *Fusion details (min-normalization, Harris, clipping) → Backup slide 20.*

## 9. BASIC's blind spot — and our idea

1:30 · cum 10:30

Here is the observation my thesis builds on. In BASIC, at no point does *\*anything\** read the image and the text *\*together\**. Each branch sees only its own half of the query. So the system can never understand that the text *\*transforms\** the image — that "as a painting" means the target IS a painting of this object, not a photo of the object near a painting.

My idea: use a multimodal large language model — a model that reads images and text jointly

— and let it *\*imagine the target\**. It looks at the query image, reads the modification, and writes one sentence describing what the target should look like. That sentence — the imagined target — is something we can embed with the same CLIP text encoder and score against every database image, as a third branch: image times text times caption.

## 10. Generating the target caption

1:30 · cum 12:00

Concretely, the captioner is InternVL, an eight-billion-parameter open multimodal model. Here is a real example: the query is this Tintin drawing plus "as an iconic rooftop sign", and the model writes: "Cartoon character in a beige coat and blue shirt appears as an iconic rooftop sign."

One practical finding: *\*how you instruct\** the model matters. I designed five instruction styles — from a strict template to a draft-and-refine strategy — and they produce noticeably different captions. Rather than betting on one, I generate all five. And note what this preserves: the caption is embedded by the same text encoder we already had. No new training, still fully zero-shot.

## 11. Captions help — immediately

1:00 · cum 13:00

Does it work? These are raw scores on the standard CLIP backbone, before any other technique. BASIC alone: roughly eighteen macro-mAP. Adding a single generated caption as a third branch: twenty-four. Averaging the five captions: twenty-five and a half — a forty-two percent relative improvement from one idea.

The averaging matters: any single prompt has quirks, and averaging five captions washes them out without having to pick a winner on the test set.

## 12. Stacking post-processing

1:30 · cum 14:30

On top of the raw scores, the BASIC paper proposes a ladder of post-processing steps, and I evaluated our caption pipeline along the same ladder. Two steps do the heavy lifting. *\*Centering\**: subtract the average database embedding — remove what all seven hundred fifty thousand images have in common, so only what is *\*distinctive\** remains. *\*Contextualization\**: enrich the text embedding with object context. Both stack nicely on both pipelines.

The last two steps — projection and query expansion — were tuned for raw CLIP features, and once the caption branch is present they consistently *\*hurt\**. So our final pipeline stops at contextualization. Being willing to drop components is part of the result.

## 13. Does it transfer? Four backbones

1:00 · cum 15:30

A fair question: is this a CLIP-specific trick? I repeated everything on four different vision-language backbones. The caption branch helps on *\*every single one\**. The most striking case is SigLIP-Large, where it *\*doubles\** macro-mAP, from twenty-one to forty-three. And notice SigLIP2 — a newer model trained with a better recipe — is the strongest backbone overall.

## 14. Final results — best backbone

1:30 · cum 17:00

So the final configuration: SigLIP2, five averaged captions, post-processing up to contextualization. The ladder has the same shape as before — peak at contextualization, projection and query expansion hurt. The headline: sixty-two macro-mAP, fully zero-shot, against fifty-seven point seven for BASIC at its own best configuration — a plus four point three gap that holds after all of BASIC's post-processing has been applied.

## 15. When it works

1:00 · cum 18:00 (flex — can compress to 0:30)

A qualitative look. Query: this pink pencil case, "without sticky paper notes but filled with pens and markers". Image similarity alone returns lookalike cases — it cannot read. BASIC does partially better. Our caption pipeline puts both true targets in the top three. The point: the caption understood the \*negation\* — "without sticky notes" — which pure visual similarity fundamentally cannot.

## 16. When it fails

1:00 · cum 19:00 (flex — can compress to 0:30)

And an honest failure. The golden mask of Agamemnon, "as a painting". The true target is an actual painting — quite abstract, visually far from the query. Every method, including ours, returns real golden masks instead: the visual identity is so strong that it overpowers the modification in the product of scores. This is the characteristic failure mode: when the modification demands a drastic domain shift, image similarity still dominates.

## 17. Limitations & future work

0:45 · cum 19:45

Three limitations, briefly. First, cost: generating captions takes about fifteen seconds per query — ninety-nine percent of the inference time — so real-time use needs smaller or quantized captioners. Second, the failure mode you just saw: drastic modifications can be overpowered by visual similarity; a fusion smarter than a plain product is a natural next step. Third, results depend on prompt design; averaging mitigates this but does not remove it.

## 18. Conclusions

0:45 · cum 20:30

To conclude. First: a specific object \*can\* be found among seven hundred fifty thousand images from an image plus a sentence, with zero training. Second, the core contribution: letting a multimodal model \*imagine the target\* and describe it is the single largest gain in the whole system, and it transfers to every backbone I tested. Third: combined with the right post-processing — and the discipline to drop the steps that stop helping — this reaches sixty-two macro-mAP, four points above the strongest baseline configuration.

Thank you — I'm happy to take questions.

## Q&A map

Metrics formulas → backup 19. Score fusion internals → backup 20. Full CLIP-L ablation table → backup 21. "The paper reports 34.35 for BASIC" → backup 22 (our reproduction of the authors' code gives 32.48; all thesis comparisons use the same reproduced code path for both methods). Latency details → backup 23. The five instruction styles with example captions → backup 24.